

Kurs-Agent — Prüfprotokoll

Stand 04.10.2026, 13:30 Uhr. Geprüft gegen ANFORDERUNGEN.md, 96 Punkte in elf Gruppen.

Der Maßstab — gesetzt am 04.10.2026, 1:30 Uhr

Die Aufgabe — gesetzt am 04.10.2026, 12:40 Uhr (Quelle: AUFGABE.md): „Ziel der Kurs-KI ist es, auf einem Ordner ein SLM oder kleines LLM speziell für die Python-Programmierung zusammen mit einem dafür passenden Agenten und den Python-Werkzeugen auf einen einzigen Ordner zu installieren. Das System soll Python-Programme auf Anforderung schreiben und selbst testen können, um Syntaxfehler über den Agenten selbst zu korrigieren, dann entweder simulieren oder direkt auf die Hardware laden. Das alles muss so geschehen, dass der Anwender nichts laden muss; es muss ohne Internetzugang funktionieren, auch alle Treiber müssen vorhanden sein. Es wird nichts installiert außer eventuell nötigen Treibern, und alles läuft nur mit den Werkzeugen auf dem besagten Ordner; es wird kein schon vorhandenes Python verwendet, keine Libs, die irgendwo gefunden wurden, nichts sonst — alles muss mitgeliefert werden. Ich will nicht sehen: ‚es muss noch nachgeladen werden‘.“

„Alles muss echt laufen auf meinem Rechner, alles inklusive LLM, Agent, Werkzeugen, Hardware. Das ist gesetzt.“ — Anweisung des Auftraggebers

Nichts gilt als fertig, was nur unter Wine, im Simulator, im Browser-Nachbau oder auf dem Linux-Prüfstand lief. Der Nachweis ist **ein vollständiger Lauf auf C:\Kurs_Agent** mit Protokoll (letzter_lauf.txt, ablage\protokoll.txt) und Bildschirmfoto — für jede der vier Säulen einzeln. Dieses Blatt führt den Stand dazu an erster Stelle, vor allem anderen:

Säule	Linux-Prüfstand	Wine	echtes Windows, Rechner des Auftraggebers
Werkzeuge (9)	☐	☐	☐ 04.10., 1:10 — 8 Aufrufe im Drehbuch, esptool/mpremote aus den mitgelieferten Rädern
Agent (Schleife, Abnahme, Nachweis, Oberfläche)	☐	☐	☐ 04.10., 1:11 — ABNAHME BESTANDEN, virtuelle LED, Nachweis „nichts verändert“
Hardware (ESP32 flashen, Programm übertragen)	—	—	☐ 04.10., 1:22 — COM3, MicroPython in 41 s, blink.py übertragen, LED blinkt

Säule	Linux-Prüfstand	Wine	echtes Windows, Rechner des Auftraggebers
Sprachmodell (llama.cpp, Qwen2.5 3B/7B)	□ 02.10., 7B fertig nach 2 Aufrufen; 04.10. 3B-Instruct und 3B-Coder	—	□ 04.10., 12:56 — Smart App Control EIN, signierter llama-server (Ollama Inc./DigiCert) lädt Qwen2.5-3B in 8 s; 23 Schritte mit echten Werkzeugaufrufen. Der Auftrag selbst wurde in diesem Lauf nicht sauber erfüllt (sieben Befunde, unten) — das Modell lief, der Agent wurde daran berichtet
Rücklesen am Gerät (esp32_nachlesen)	—	—	□ 04.10., 13:17-13:19 — COM3: 1-Hz-Programm 1,04 Hz BESTANDEN; Gegenprobe gegen 2 Hz NICHT BESTANDEN; 2-Hz-Programm aufgespielt, 2,10 Hz gemessen; zurück auf 1 Hz, 1,04 Hz. Die LED blinkt, und das Gerät sagt es selbst

Alle Säulen tragen auf dem echten Rechner □ (04.10.2026, 13:19 Uhr). Offen ist nicht mehr, **ob** die Kette läuft, sondern wie gut das Modell sie führt — siehe den Lauf von 12:56 und die Messreihe auf dem Prüfstand. Ein vollständiger Lauf mit dem berichtigten Agenten auf C:\Kurs_Agent steht als Nächstes an (Agent neu starten, Karte 2 mit Hardware).

Womit geprüft wurde

Umgebung	was darin lief
Linux (20 Kerne, 67 GB)	pruefe_alles.py (14 Prüfungen), agent/pruefe_schleife.py (8 Punkte), echte Agentenläufe mit Qwen2.5-7B über llama.cpp

Umgebung	was darin lief
Windows unter Wine 10.0 (eigener Bereich, kein fremdes Präfix angefasst)	das mitgelieferte Windows-Python 3.12.10, <code>START.bat</code> über <code>cmd</code> , alle neun Werkzeuge, <code>pip</code> , <code>esptool</code> -Bau aus Quelltext, die Weboberfläche
Browser (Chromium über Playwright)	die Anleitungsseite, die Weboberfläche, die virtuelle LED

Grenze, die dazugehört: Wine ist nicht Windows. Es bildet das Verhalten nach, nicht jede Eigenheit. Was hier läuft, läuft sehr wahrscheinlich auch dort — bewiesen ist es erst auf einem echten Windows-Rechner. Die Punkte, die davon berührt sind, stehen unten unter „offen“.

Seit 04.10.2026 wird nicht mehr unter Wine geprüft (Grundsatz 33): Wine-Starts füllten zweimal die Prozesstabelle des Arbeitsbereichs, und der echte Windows-Lauf fand vier Fehler, die Wine nicht zeigte. Windows-Dinge laufen jetzt auf dem Rechner des Auftraggebers über die Brücke; die Wine-Spalte oben ist Vorgeschichte. Eine eigene Windows-VM folgt, sobald der Admin sie einrichtet.

A — Autarkie

	Anforderung	geprüft	Ergebnis
A1	Paket bringt alles mit	<code>pfade.bericht()</code> unter Wine	erfüllt — Python, <code>pip</code> , 25 Paketdateien, Treiber gefunden; Modell fehlt noch (nicht geladen)
A2	im Kurs nichts aus dem Netz	Netzwächter (<code>socket.connect</code>) während eines vollen Laufs	erfüllt — keine Verbindung nach außen; Gegenprobe mit <code>pypi.org</code> und 1.1.1.1 schlug an
A3	nichts installiert	Durchsicht + <code>nachweis_autark</code> unter Wine	erfüllt — kein Installationsprogramm, keine Registrierung, keine Adminrechte
A4	im Kursbetrieb kein fremdes Python	<code>pruefe_alles.py</code> A4	erfüllt — keine Suche außerhalb des Füllzweigs; das gefundene Python nur für <code>hole_paket.py</code> und die <code>pip</code> -Probe. Gegenprobe: ein <code>where</code> an anderer Stelle eingesetzt — die Prüfung nannte die Zeilennummer
A5	kein fester Pfad	<code>pruefe_alles.py</code> A5	erfüllt — Lauf aus <code>C:\Stick</code> ohne Änderung
A6	ohne Modell läuft der Rest	<code>agent.py --drehbuch</code> unter Wine	erfüllt — alle sieben Schritte, Abnahme bestanden

	Anforderung	geprüft	Ergebnis
A7	Doppelklick genügt auch beim ersten Mal	START.bat auf leerem Ordner unter Wine, vier Fälle	erfüllt — siehe unten

Zu A7 — vier Fälle am leeren Ordner geprüft:

Fall	Ergebnis
kein Python auf dem Rechner	nennt zwei Wege: gefüllten Ordner kopieren, oder Python installieren
Python gefunden, aber ohne pip	erkannt und abgewiesen — „darin läuft kein pip“, mit dem Hinweis auf den Store-Platzhalter
Python mit pip, Antwort n	„Abgebrochen — es wurde nichts geholt und nichts geändert“
Python mit pip, Eingabetaste	füllt, und läuft danach weiter, als wäre gerade gestartet worden

Dabei gemessen: PIP_CACHE_DIR zeigte auf C:\LeerTest\ablage\pip_zwischenspeicher, PYTHONNOUSERSITE auf 1 — der Zwischenspeicher des Benutzers bleibt also auch beim Füllen unberührt.

Der erste Lauf fiel durch: ein verdoppeltes Caret (2^^>nul statt 2^>nul) zerriss den Batch-Block. Die Folge war schlimmer als ein Abbruch — das Skript übersprang den Füllzweig **stillschweigend** und meldete danach ein Python, das es nicht gab. Durch Lesen war das nicht zu sehen.

B — keine Störung dessen, was schon da ist

	Anforderung	geprüft	Ergebnis
B1	fremdes Python unberührt	A4	erfüllt
B2	fremde site-packages unberührt	nachweis_autark vorher/nachher unter Wine	erfüllt — „Nichts außerhalb des Paketordners hat sich verändert“
B3-5	Benutzerpakete, pip-Cache, PYTHON-Variablen abgeschirmt	pruefe_alles.py B3-5	erfüllt
B6	belegter Port wird nicht übernommen	Port 8777 besetzt, Oberfläche gestartet	erfüllt — wich auf 8778 aus
B7	belegter COM-Anschluss	—	offen (keine Hardware)
B8	zwei gleichzeitige Läufe	pruefe_alles.py B8	erfüllt — der zweite wird abgewiesen
B9	eine fremde Umgebung stört uns nicht	PYTHONPATH, PYTHONHOME, PIP_INDEX_URL, PYTHONSTARTUP, PYTHONUSERBASE gesetzt	teilweise — siehe unten

Zu B9: PIP_INDEX_URL, PYTHONSTARTUP und PYTHONUSERBASE stören nicht. Bei PYTHONPATH wird ein fremdes Modul **einmal geladen**, bevor eine eigene Zeile läuft; der eingebaute Selbst-Neustart (agent/sauber.py) sorgt dann für einen sauberen Durchgang — die Abnahme stimmte. PYTHONHOME verhindert, dass Python überhaupt startet; das kann kein Python-Code heilen, deshalb räumt START.bat beide Variablen vor dem Start weg.

C — nur im Kursordner schreiben

	Anforderung	geprüft	Ergebnis
C1	Wächter meldet Schreibversuche draußen	pruefe_alles.py C1	erfüllt — meldet den verbotenen, lässt den erlaubten durch
C2-3	Ausbruch und absolute Pfade abgewiesen	pruefe_alles.py C2-3	erfüllt — 3 von 3 abgewiesen
C4	nichts außerhalb während eines Laufs	nachweis_autark unter Wine	erfüllt
C5	paket\ bleibt unverändert	pruefe_alles.py C5	erfüllt

Zu C4: Ein Vergleich des *gesamten* Dateisystems vorher/nachher war zu stumpf — er fand 19 Änderungen, alle von fremden Programmen (Entwicklungsumgebung, andere Sitzungen). Auf einem benutzten Rechner ist „nichts hat sich geändert“ nicht erreichbar. Deshalb misst der Wächter am Verursacher statt am Rechner.

D — restlos entfernbar

	Anforderung	geprüft	Ergebnis
D1-2	löschen und nachzählen	pruefe_alles.py D1-2 + Wine	erfüllt — 3778 von 3783 Dateien gelöscht
D3	Modellserver wird vorher beendet	Gegenprobe mit Probeprozess	erfüllt
D4	Rechner im Ausgangszustand	nachweis_autark	erfüllt
D5	beliebig wiederholbar	drei Zyklen, Prüfsumme über paket\	erfüllt — b7ea1793... vorher wie nachher
D6	Selbstlöschung der Kopie	—	offen (Betriebsart [2] nicht unter Wine erprobt)

Zu D1: Die letzten fünf Dateien sind das Python, mit dem das Aufräumen gerade läuft — Windows hält python.exe und die DLLs fest. Das Werkzeug meldet das jetzt ehrlich und hinterlässt eine Marke; START.bat entfernt den Rest, sobald Python beendet ist. Nachgeprüft: danach blieb nichts.

E — Weitergabe

	Anforderung	geprüft	Ergebnis
E1	Stick und Platte gleichermaßen	Lauf aus C:\Stick	erfüllt
E2	baue_stick.py ohne Entwicklungsreste	ausgeführt	erfüllt — 98 Dateien, keine Arbeitsordner
E3	Fehlendes wird gemeldet	pfade.bericht(), baue_stick.py	erfüllt — „Modell FEHLT“ wird genannt
E4	Modellwahl nach freiem Speicher	Gegenproben bei 52 / 7,5 / 5 / 3 / 1 GB	erfüllt — 7B / 7B / 3B / keines / keines

F — Bedienung

	Anforderung	geprüft	Ergebnis
F1	Doppelklick genügt	START.bat unter cmd	erfüllt — Menü erscheint, Python wird bereitgelegt
F2	Aufgaben anklickbar	Oberfläche im Browser	erfüllt — vier Karten
F3	jeder Schritt sichtbar	Oberfläche	erfüllt — Werkzeug, Werte, Ergebnis, Urteil
F4	gesperrte Knöpfe erklärt	Oberfläche	erfüllt — Ampel zeigt „läuft ...“
F5	Fehlermeldung nennt den nächsten Schritt	Durchsicht aller Meldungen	erfüllt
F6	Löschen wird bestätigt	Oberfläche	erfüllt — Rückfrage
F7	Anleitung und START.bat nennen dieselben Nummern	pruefe_alles.py F7	erfüllt — 9 Verweise stimmen; Gegenprobe mit verfälschter Nummer schlug an

G — der Lehrinhalt

	Anforderung	geprüft	Ergebnis
G1	schreiben, prüfen, erst dann ausliefern	echter Lauf mit 7B	erfüllt
G2-3	Abnahme besteht bei richtig, fällt bei falsch durch	pruefe_alles.py G2-3	erfüllt — vier Fälle
G4	FERTIG gilt nicht nach fehlgeschlagener Prüfung	pruefe_schleife.py	erfüllt — 8 von 8
G5	Fehler ist Nachricht, nicht Abbruch	echter Lauf	erfüllt — NICHT BESTANDEN → berichtet → BESTANDEN
G6	geprüfter Quelltext bleibt unverändert	pruefe_alles.py G6	erfüllt

	Anforderung	geprüft	Ergebnis
G7	Erwartung aus dem Auftrag, nicht vom Modell	pruefstand/pruefe_rege	erfüllt — 0,5 Hz des Modells durch 1,0 Hz des Auftrags ersetzt, Hinweis im Ergebnis (04.10., 13:10)
G8	ungeprüfte Änderung oder durchgefallene Prüfung → kein FERTIG; nach drei Abweisungen NICHT ABGENOMMEN	pruefstand/pruefe_unge 5 Punkte	erfüllt .py FERTIG genau einmal angenommen (nach bestandenem Test); dreimal FERTIG ohne Test → „Nicht abgenommen“, kein FERTIG im Protokoll
G9	nur in diesem Lauf Geschriebenes wird geprüft	pruefe_regeln.py	erfüllt — fremd.py abgewiesen
G10	Aufruf ohne WERKZEUG: erkannt, Satzmitte nicht	pruefe_regeln.py	erfüllt — 4 Fälle
G11	aufraeumen nicht fürs Modell	pruefe_regeln.py	erfüllt — abgewiesen, Lauf endet regulär, Ablage bleibt (Prüfstand 11:21: das 3B-Modell hatte es als 9. Schritt gerufen, Server weg, Protokoll weg)
G12	Coder-Modell im Paket	Messreihe 2 (unten), modell.waehlen	erfüllt — Coder-3B nach C:\Kurs_Agent\paket\model\ übertragen (04.10., 13:40), wirksam nach Neustart des Agenten

H — Hardware

	Anforderung	geprüft	Ergebnis
H1-H3	fabrikneu oder vorprogrammiert, Antwort vor dem Schreiben, kein Raten	Werkzeugtexte, Abbruch ohne Antwort	teilweise — die Logik greift, der Fall ohne ESP32 ist geprüft
H4	Treiber im Paket mit Prüfsummen	paket\treiber\LIESMICH	erfüllt — CH341SER, CH343SER; CP210x fehlt (Silabs sperrt diese Umgebung)
H5	der Agent installiert keinen Treiber	Durchsicht	erfüllt — es gibt kein Werkzeug dafür
H6	ohne Hardware bis zur Abnahme	voller Lauf unter Wine	erfüllt

	Anforderung	geprüft	Ergebnis
H7	vom Gerät zurücklesen	esp32_nachlesen an COM3, über die Brücke ausgelöst, 7 Schritte	erfüllt — 1,04 Hz / 2,10 Hz / 1,04 Hz, Gegenproben durchgefallen wo sie mussten (04.10., 13:17-13:19)
H8	Rücklesen mit Flankeninterrupt und Mikrosekundenzähler des Geräts statt Abtastschleife („die Quarzfrequenz ist doch bekannt“)	Laptop 20:15, 1-s/1-s-Programm	erfüllt — 2,0000 s zwischen Einschaltvorgängen, 0,500 Hz (vorher per Abtastung 1,937 s, 0,52 Hz)
H9	esp32_uebertragen liest nach dem Neustart selbst am Gerät nach, wenn eine Erwartung bekannt ist („das muss man doch durch Rücklesen rausfinden“)	Werkzeugbeschreibung, Agent reicht die Erwartung durch	eingebaut , am Gerät noch nicht einzeln gemessen
H10	Zweiter unabhängiger Weg für das Verhalten nach dem Neustart: Programm meldet jedes Schalten seriell, die Brücke liest nur mit (Öffnen = Boot)	20:08, blink_diag.py	erfüllt — „AN n“ alle 500 ms fortlaufend; die Beobachtung „1 Hz“ war nicht das Gerät

I — Ferndiagnose

	Anforderung	geprüft	Ergebnis
I1	ein Befehl, eine Datei, sechs Abschnitte	sammle_bericht.py auf Linux	erfüllt — 9 kB: Rechner, Paket, Selbstprüfung, Werkzeugprobe, Protokoll, Ablage
I2	kein Benutzer-, kein Rechnername	Gegenprobe: Rechnername im Bericht gesucht	erfüllt — nicht gefunden
I3	Fehlendes steht im Bericht, statt ihn scheitern zu lassen	Lauf ohne Modell, mit nicht startbarem Python	erfüllt — „Das eigene Python ließ sich nicht starten (PermissionError). Der Bericht wurde deshalb mit /usr/bin/python3 erstellt.“

I5 | Fehlerbericht bei Abbruch, Protokoll endet mit ABBRUCH/ENDE | `pruefe_regeln.py`, erzwungener `RuntimeError` | **erfüllt** — `fehlerbericht.txt` mit Rückverfolgung und 40 Protokollzeilen | I6 | volles Kontextfenster beendet den Lauf nicht | `pruefe_regeln.py`, `KontextVoll` am ersten Schritt | **erfüllt** — zusammengefasst, Lauf endet mit FERTIG |

Zu I3: Das Werkzeug entstand am 02.10.2026 um 09:57, unmittelbar vor dem Abbruch der Sitzung, und war bis zur Wiederaufnahme **nie ausgeführt worden**. Beim ersten Lauf zeigte sich, dass es richtig arbeitet — und dass es im Anforderungsblatt gefehlt hatte. Gruppe I ist deshalb nachgetragen; aus 47 Punkten wurden 50.

J — das Gespräch

	Anforderung	geprüft	Ergebnis
J1	Kopf oben, Verlauf scrollt, Eingabe unten, nächste Anweisung immer möglich	Playwright gegen die Oberfläche auf dem Prüfstand (Coder-3B)	erfüllt — Lage der drei Bereiche gemessen, Eingabe nach dem Ende frei, Verlauf folgt bis unten (Abstand 0 px), 0 Skriptfehler
J2	Zusammenhang über Anweisungen, ein Modellstart	<code>pruefe_gespraech.py</code> 19/19	erfüllt — sechs Anweisungen, Modellserver einmal gestartet
J3	anderer Pin / andere Frequenz aus einem Satz	<code>pruefe_gespraech.py</code> ; Browser mit Coder-3B: 1 Hz → „Nimm jetzt GPIO 4 statt 2“ → „10 Hz“	erfüllt — Erwartung {pins [4], 1 Hz} dann {pins [4], 10 Hz}; 10 Hz erst NICHT BESTANDEN, mit vorgerechneter Pause (0,05 s) BESTANDEN
J4	erst prüfen, dann laden	<code>pruefe_gespraech.py</code> Anweisung 4	erfüllt — ungeprüfte Fassung wird nicht übertragen
J5	Anschluss vom Menschen	<code>pruefe_gespraech.py</code> ; Browser: Modell wollte COM5	erfüllt — „COM5 wurde vom Menschen nicht genannt — geraten wird nicht“
J6	eigenes Programm wörtlich	<code>pruefe_gespraech.py</code> Anweisung 5	erfüllt — Inhalt byteweise gleich, Modell prüft nur
J7	mehrere Aufrufe, Aufruf ohne JSON, Behauptung im FERTIG	<code>pruefe_regeln.py</code> 27/27	erfüllt
J8	Neuladen ohne Neustart	dreimal am 04.10. angewendet	erfüllt — /neuladen meldet die Dateizeiten; die Seite wird je Aufruf gelesen

	Anforderung	geprüft	Ergebnis
J9	Agent prüft selbst nach FERTIG ohne Prüfung	pruefe_ungeprueft.py 6/6; Browser mit Coder-3B	erfüllt — 13:04 Prüfstandzeit: viermal geschrieben, nie geprüft → „Nicht abgenommen“; nach der Korrektur 13:09: Selbstprüfung gegen pins [4], BESTANDEN, FERTIG
J10	Quereingabe während des Laufs (Zwischenruf), „com 3“ = COM3	pruefe_gespraech.py 24/24	erfüllt — Eingabefeld bleibt offen, Zwischenruf wird beim nächsten Schritt aufgenommen; Anlass 19:57: „nein es ist com 3 nicht com 5“ wurde nicht erkannt
J11	Behauptung und Verlangen je Anweisung: „lade“, „lies nach“ müssen in dieser Anweisung gelaufen sein	pruefe_gespraech.py 5b	erfüllt — Anlass 20:10: „erfolgreich auf dem ESP32 geladen“ ohne Übertragung in dieser Anweisung
J12	Live am Laptop, 20:01-20:04 Uhr (Coder-3B): blink.py 2 Hz (erst 1 Hz durchgefallen, berichtigt, bestanden); Modell wollte COM5, abgewiesen, fragt; „Der Anschluss ist COM3. Lade ... und lies nach“: esp32_uebertragen COM3, esp32_nachlesen 2,06 Hz BESTANDEN, ehrliches FERTIG; Taschenrechner (tkinter, 37 Zeilen) als Fenster offen, Nutzer: „der Taschenrechner funktioniert“	Gespräch über die Brücke, Nutzer am Bildschirm	erfüllt

	Anforderung	geprüft	Ergebnis
J13-J18	ergebnisoffen, Methode bereitstellen, Fokus, fehlende Datei, Anmerkung, Erwartung aus Worten	pruefe_gespraech.py 30/30, Laptop-Gespräche 05.10.2026 9:09-10:20	erfüllt — „hallo bist du bereit?“ → „Ja, ich bin bereit“; erfundene Arbeit („geschrieben, geprüft, übertragen“ ohne Werkzeug) wird als unwahr zurückgewiesen, Fokus liefert den Aufruf; keine Behauptung ging als abgenommen durch

K — allgemeine Python-Programme

	Anforderung	geprüft	Ergebnis
K1	programm_ausfuehren: Ausgabe, Fehler, Rückgabewert, Eingabe	Prüfstand Linux: rechne.py (2+3=5, BESTANDEN), kaputt.py (ZeroDivisionError, NICHT BESTANDEN), lies.py mit Eingabe 21 → 42	erfüllt
K2	Fensterprogramm starten und beobachten	Laptop 14:42 über die Brücke: gui_probe.py	erfüllt — „Nach 3,0 s läuft es noch; Fenster: ‘Fensterprobe Kurs-Agent’“, LAUF BESTANDEN
K3	tkinter im Paket, ins eigene Python entpackt	umgebung_anlegen am Laptop	erfüllt — 944 Dateien, „Gegenprobe tkinter: Tk 8.6“, Python 3.12.10 (python.org) mit _tkinter.pyd aus conda-forge
K4	Lauf zählt als Prüfung	pruefe_regeln.py	erfüllt — kaputter Lauf sperrt FERTIG, guter gibt frei
K5	PWM-Hüllkurve, Form als Kriterium	pruefe_pwm.py 7/7; Laptop 10:17: Modellprogramm sinusförmig 0,97 Hz erkannt	erfüllt
K6	gedachte Uhr im Nachbau	pruefe_pwm.py, pruefe_alles.py 19/19	erfüllt — Läufe in 0,6 s statt 4 × 4 s; Takt unabhängig von der Windows-Uhr

Der Vormittag des 05.10.2026 im Gespräch (Coder-3B, Laptop): Begrüßung und Fragen in Worten beantwortet (Hardwarefrage falsch: „LED am Bildschirm“); Blinkauftrag zunächst

mit erfundener Arbeit beantwortet (viermal „geschrieben, geprüft, übertragen“ ohne ein Werkzeug) — vom Agenten abgefangen, danach Fokus eingebaut; Atem-Aufgabe: 1. Anlauf PWM-Trägerfrequenz als Takt (1000 Hz) festgehalten, Dreieck statt Sinus; 2.-4. Anlauf deckten Lücken im Prüfwerkzeug auf (keine Hüllkurve, math verboten, Windows-Uhr, stilles blink.py, Wechselzahl bei PWM); 5. Anlauf läuft mit allen Korrekturen. Messreihe 3 (Coder-3B gegen Coder-7B, acht Anweisungen, derselbe Agent) läuft auf dem Prüfstand.

K erfüllt am Laptop, 20:05 Uhr: „Programmiere einen einfachen Taschenrechner rechner.py mit grafischer Oberfläche“ ergab 37 Zeilen tkinter; programm_ausfuehren mit fenster und offen lassen; Fenster „Taschenrechner“ 320×239 erkannt; Nutzer: „der Taschenrechner funktioniert“.

Der erste Lauf mit Modell auf Windows — 04.10.2026, 12:56 bis 13:04

Smart App Control EIN. Der signierte llama-server aus dem Ollama-Archiv lädt Qwen2.5-3B-Instruct in 8 s. Karte 1 („LED blinken lassen, nur geprüft“), über die Brücke gestartet, aus dem Arbeitsbereich mitgelesen. 23 Schritte, dann HTTP 400 vom Server. Was der Lauf zeigte, und was daraus wurde — jede Korrektur hat eine Gegenprobe in pruefstand/pruefe_regeln.py oder pruefe_ungeprueft.py:

Befund im Lauf	Korrektur	Anforderung
Schritte 1-3: das Modell schrieb schreib_datei {...} ohne WERKZEUG;; nichts wurde ausgeführt	Name eines bekannten Werkzeugs am Zeilenanfang mit JSON gilt als Aufruf	G10
Schritt 4: programm_testen blink.py prüfte eine alte blink.py aus der Nacht (1 Hz) — und bestand	geprüft wird nur, was in diesem Lauf geschrieben wurde	G9
Schritte 17-21: das Modell schrieb die Erwartung von 1,0 auf 0,5 Hz um	Erwartung kommt aus dem Auftrag und ist nicht verhandelbar	G7
Schritt 19/22: FERTIG mit falscher Begründung („0,5 s + 0,5 s = 1 Hz, nicht 0,5 Hz“)	wird durch G7/G8 gegenstandslos; FERTIG nach durchgefallener Prüfung wird weiter abgewiesen (G4)	G4
Schritt 23: HTTP 400 — Kontextfenster 8192 voll (jeder Testlauf liefert >120 Zeilen Zeitachse)	Kontext 16384; Werkzeugergebnisse gekürzt ins Modellgedächtnis; bei KontextVoll zusammenfassen statt abbrechen	I6
Protokoll endete 13:03:43, der Abbruch um 13:04:18 stand nirgends	fehlerbericht.txt, Protokollzeilen ABBRUCH/FEHLERBERICHT/ENDE	I5
Prüfstand 11:14: dritte Fassung geschrieben, nicht geprüft, FERTIG angenommen	ungeprüfte Änderung sperrt FERTIG	G8
Laptop Schritt 9 / Prüfstand 11:21: das Modell rief aufräumen mitten im Auftrag — Server beendet, Ablage und Protokoll gelöscht, „Connection refused“	aufräumen nicht in der Werkzeugliste des Modells; Aufruf wird abgewiesen	G11

Befund im Lauf	Korrektur	Anforderung
Startfenster warnte „llama.cpp ist nicht signiert“ — stimmte nicht mehr	pruefe_rechner.py fragt die Datei nach ihrer Signatur	—
jeder Testlauf öffnete einen neuen Browser-Reiter „Virtuelle LED“	unter der Oberfläche kein eigener Reiter mehr; die Oberfläche zeigt die LED selbst	F

Messreihe Prüfstand (Linux, 20 Kerne), Karte 1, alter Agent: Qwen2.5-3B-Instruct 3 × schreiben, 2 × prüfen (1 durchgefallen, 1 bestanden), FERTIG nach 83 s — mit ungeprüfter dritter Fassung. Qwen2.5-Coder-3B-Instruct 2 × schreiben, 2 × prüfen (1 durchgefallen, 1 bestanden), FERTIG nach 67 s, geprüfte Fassung geliefert.

Messreihe 2 (berichtigter Agent, 11:25-11:33 Prüfstandzeit = 13:25-13:33):

Modell	Karte 1 „blink.py, 1 Hz“	Karte 3 „zwei.py, zwei LEDs“
Qwen2.5-3B-Instruct	bestanden , aber 16 Werkzeugaufrufe, 95 s: nach der bestandenen Prüfung noch 14 Aufrufe ins Leere (ports, esptool, pipx, requirements.txt, COM5 ...)	nicht bestanden: Datei „two.py“ statt zwei.py, 2 × Syntaxfehler, 3 × Absturz, 4. Fassung ungeprüft, FERTIG beim zweiten Anlauf durchgerutscht (seither: drei Abweisungen, dann NICHT ABGENOMMEN)
Qwen2.5-Coder-3B-Instruct	bestanden , 4 Werkzeugaufrufe, 83 s: schreiben → NICHT BESTANDEN → berichtigen → BESTANDEN → FERTIG	nicht bestanden: Datei „two.py“, Programm blockiert in der ersten Blinkschleife, Pin 4 nie geschaltet — dreimal dieselbe Diagnose, nicht berichtigt

Folgerung: Karte 1 kann das Coder-3B sauber, das Instruct-3B nur mit Umwegen; Karte 3 (zwei Takte gleichzeitig, nicht blockierend) überfordert beide 3B-Modelle — sie gehört zum 7B oder wird als „schwere Karte“ ausgewiesen. Das Coder-Modell ist seither die Wahl (G12); beide Modelle nannten die Datei anders als der Auftrag, der Agent weist jetzt darauf hin.

Die scharfe Prüfung vom 02.10.2026: Bleibt der Zielrechner unversehrt?

Auf Weisung eigens geprüft, mit pruefe_unversehrtheit.py — drei Wege, die einander nicht glauben müssen:

Weg	Ergebnis
Durchsicht jeder schreibenden Zeile	52 gefunden, alle führen in den Kursordner. Das Werkzeug löst dafür Namen auf, statt zu raten: PID_DATEI → ABLAGE → WURZEL.
Verbotsliste: 15 Befehle, die Windows verändern	keiner kommt vor — keine Registrierung, kein setx, kein Dienst, keine Aufgabenplanung, keine Rechteänderung

Weg	Ergebnis
Messung eines echten Laufs	159 933 Dateien außerhalb des Kursordners aufgenommen: 0 neu, 0 verschwunden, 1 verändert — und das eine ist <code>vscode.lock</code> der Entwicklungsumgebung, nicht des Pakets

`setlocal` steht im Startskript: die 24 gesetzten Variablen wirken nur im eigenen Fenster.

Vier Fehler hat diese Prüfung gefunden — zwei im Paket, zwei in der Prüfung selbst:

	Fehler	Folge
1	<code>esp32_firmware</code> lud fehlende Firmware aus dem Netz und legte sie nach	verletzte A2 (kein Netz im Kurs) und C5 (<code>paket\</code> wird nur gelesen). Entfernt: fehlt sie, wird das gesagt
2	<code>paket\firmware\umgebung_anlegen</code> tat dasselbe mit Python und <code>get-pip</code>	dito, entfernt
3	<code>adafruit-ampy</code> stand auf der Erlaubnisliste, lag aber nicht im Vorrat	das Modell liest diese Liste und hätte es anfordern können; im Kurs ohne Netz wäre es gescheitert. Gestrichen, und E5 prüft das jetzt
4	Die Prüfung selbst übersah <code>subprocess.run(["reg", "add", ...])</code>	ein verbotener Befehl in Listenform blieb unsichtbar. Behoben; die Gegenprobe findet ihn jetzt

Dazu drei Befunde, die bleiben und benannt sind:

- Der Wächter lässt `%TEMP%` ausdrücklich zu — Python arbeitet dort selbst. Steht jetzt als zweite Ausnahme in Gruppe C.
- Die Wächterprobe schreibt absichtlich einmal ins Benutzerverzeichnis, um zu zeigen, dass der Wächter anschlägt. Sie räumt in jedem Fall wieder auf, auch bei einem Abbruch.
- Die Aussage „eine einzige Stelle startet Programme“ stimmte nicht: es sind fünf. Alle fünf reichen dieselbe abgeschirmte Umgebung weiter — **B10** prüft das und fällt durch, sobald eine es vergisst.

Was dabei an Prüfungen dazukam

	prüft	Gegenprobe
B10	jeder Programmstart reicht	<code>env=</code> an einer Stelle entfernt
E5	<code>env=umgebung()</code> weiter	→ Zeilennummer gemeldet
	jedes erlaubte Paket liegt bei, und jede geltende Abhängigkeit	ein Rad aus dem Vorrat genommen → gemeldet
I4	die Unversehrtheitsfrage lässt sich selbst beantworten	Spur ins Benutzerprofil und <code>setx</code> eingebaut → beide als schwerwiegend gemeldet

Zu E5: Die Bedingungen sind der Kern. `typing-extensions` steht in `cryptography`, aber nur für `python_full_version < '3.11'`; das Paket bringt 3.12.10 mit, die Zeile gilt also nicht. Wer sie mitzählt, jagt einem Phantom nach — wer alle Bedingungen ignoriert, übersieht die echten. Geprüft: 25 Pakete, 5 erlaubte, 13 geltende Anforderungen, **jede liegt bei**.

Fehler, die diese Prüfung gefunden hat

Zwölf, keiner davon war durch Lesen zu sehen.

Nur unter Windows sichtbar

	Fehler	Folge
1	<code>sys.path</code> hat beim eingebetteten Python nur zwei Einträge	Kein Werkzeug lief. „No module named <code>_muster</code> “
2	<code>umgebung_anlegen</code> sprang bei vorhandenem Python heraus	<code>pip</code> wäre nie eingerichtet worden
3	<code>Cannot import 'setuptools.build_meta'</code>	<code>esptool</code> ließ sich offline nicht bauen
4	<code>colorama</code> fehlte im Paket	<code>pip download</code> wertet Windows-Marker nach dem laufenden System aus
5	Extras abgeschnitten (<code>esp-pylib[cli,ide,serial]</code>)	<code>websockets</code> fehlte
6	<code>machine.py</code> unauffindbar — <code>._pth</code> ignoriert Arbeitsverzeichnis und <code>PYTHONPATH</code>	der Prüfschritt lief nicht
7	<code>for %%Z in (muster)</code> findet nichts	„Das Paket ist nicht gefüllt“, obwohl es gefüllt war
8	Expand-Archive braucht PowerShell	Auspacken scheiterte; jetzt liegt Python ausgepackt im Paket
9	Sonderzeichen in <code>START.bat</code>	G00 statt <code>—</code> ; die Datei ist jetzt reines ASCII
10	Ausgabe mit cp1252 gelesen, UTF-8 geschrieben	<code>UnicodeDecodeError</code> im Nebenlauf → leere Ausgabe bei Rückgabewert 0 : Das Werkzeug lief, niemand erfuhr etwas

In beiden Umgebungen

	Fehler	Folge
11	<code>nachweis_autark</code> nahm <code>sys.executable</code> als „Python des Rechners“	Es maß unseren eigenen Ordner — ein beruhigendes Ergebnis ohne Bedeutung
12	Die Oberfläche beurteilte das Ergebnis ein zweites Mal, unvollständig	Bei bestandener Abnahme fehlte die Marke. Die Beurteilung steht jetzt an einer Stelle

Dazu: Das Aufräumen kann sich nicht selbst löschen; webbrowser.open riss die Oberfläche mit; PYTHONPATH lud fremden Code in unseren Prozess.

Was offen bleibt

	warum
Lauf auf echtem Windows	Wine bildet nach, beweist aber nicht. Dies ist der wichtigste offene Punkt
B7, H1-H3 am Gerät	kein ESP32 vorhanden
D6 Selbstlöschung	Betriebsart [2] nicht erprobt
CP210x-Treiber	Silicon Labs sperrt diese Umgebung; auf einem gewöhnlichen Anschluss lädt hole_paket.py ihn mit
Lauf mit Modell unter Windows	das Modell wurde in den Wine-Bereich nicht geladen (4,7 GB)

Prof. Dr.-Ing. Ralph Wystup M.Sc. — erstellt mit KI und Agent (Claude Code, Anthropic)